

Deep Research Digest: Real-Time Sample-Based Audio for a Mobile Rhythm Game

Executive Summary

This digest provides a comprehensive technical framework for developing a piano-tiles-style rhythm game on Unity for iOS and Android, focusing on the core challenge of real-time, sample-based music rendering. The analysis concludes that success hinges on a series of strategic decisions made early in development to mitigate significant technical and legal risks.

The primary technical risk is audio latency, particularly on the fragmented Android ecosystem. Standard Unity audio proves insufficient for the low-latency demands of a rhythm game, with delays on some devices rendering the experience unplayable. The strongest mitigation strategy is the adoption of audio middleware, such as FMOD or Wwise, which provides a more direct and consistent low-latency path to the hardware. Prototyping with middleware from day one is a critical de-risking step.

The sample-set strategy represents a core design trade-off between audio fidelity, application size, and memory footprint. A recommended approach for mobile is a "sparse but smart" sample set of 50-70 notes, covering the most-used 5-6 octaves of the piano. This minimizes the need for aggressive, artifact-prone pitch-shifting. To achieve realism, this base set must be augmented with 2-4 velocity layers to capture dynamic changes in timbre and 2-4 round-robin samples per layer to avoid mechanical repetition. For mobile deployment, the ADPCM codec offers the best balance of low decoding latency and reasonable compression for the hundreds of short note samples.

Content sourcing requires rigorous legal diligence. The term "public domain" is complex; the copyright of a musical composition is separate from the copyright of a specific published edition (score) and a specific performance (recording or MIDI file). The safest path for commercial use is to create game charts from scratch, using scores of pre-1929 editions found on reputable archives like the International Music

Score Library Project (IMSLP).

Finally, the playback architecture must be built for extreme performance. All note-playing AudioSource components must be managed by a robust object pool to eliminate runtime instantiation and garbage collection, which would otherwise cause fatal gameplay stutters. This pool must incorporate a musically intelligent voice-stealing algorithm to gracefully manage polyphony under the pressure of rapid, sustained classical passages, ensuring the game remains performant and sonically coherent across a range of mobile devices.

Section 1: Strategic Sample-Set Design for Mobile Pianos

The foundation of the game's audio experience is the piano sample library. Crafting this library for mobile platforms is a delicate balancing act. The goal is to achieve a sound that is rich, expressive, and authentic to classical piano performance, while adhering to the strict constraints of mobile storage, RAM, and CPU. This section dissects the trade-offs involved and outlines a strategy for creating an optimized yet high-quality sample set.

1.1 The Mobile Piano Trilemma: Balancing Sample Count, Quality, and Footprint

The core challenge lies in the trilemma between the number of individual samples, the perceived audio quality, and the resulting storage and memory footprint. Historically, hardware samplers were severely limited, with 128 MB considered a small capacity even for professional use, and 512 MB being a high-end feature.¹ While modern mobile devices are more capable, the constraints of app store download limits and user device storage remain paramount.

Analysis of Sample Counts

The number of unique piano notes sampled directly impacts the final asset size and the degree to which artificial techniques like pitch-shifting must be employed.

- **Full 88-Key Set:** Sampling every key of a standard piano provides the highest level of authenticity, as no pitch-shifting is required. However, the data footprint is prohibitive for a mobile game. Using a conservative estimate of a 3-second,

16-bit, 44.1 kHz mono sample per key (approx. 264 KB), an 88-key set would consume over 23 MB of RAM just for a single velocity layer, uncompressed.² This size is multiplied by the number of velocity layers and round-robin variations, quickly scaling into hundreds of megabytes.

- **70+ Sample Set:** This approach, covering roughly six octaves (e.g., C1 to C7), represents a strong compromise. It aligns with the range used in many high-quality digital pianos and covers the vast majority of the repertoire from the Baroque, Classical, and early Romantic periods.⁴ It significantly reduces the need for aggressive pitch-shifting while still carrying a notable, but more manageable, storage cost.
- **30–50 Sample Set:** This is the most pragmatic and common strategy for mobile development. It typically involves sampling one note every few semitones (e.g., every minor third, or C, D#, F#, A). The notes in between are generated by pitch-shifting the nearest recorded sample. This dramatically reduces the asset footprint but places immense importance on the quality of the pitch-shifting algorithm and the acceptable range of transposition to avoid audible artifacts.⁶

1.2 Bridging the Gaps: Advanced Sampling Techniques for Realism and Efficiency

A sparse sample set can only sound realistic if augmented with techniques that simulate the complex behaviors of an acoustic piano. These techniques add complexity and increase the total sample count but are essential for moving beyond a flat, lifeless sound.

1.2.1 The Limits of Pitch-Shifting and Quality Preservation

Pitch-shifting is the primary tool for filling the gaps in a sparse sample set. However, it is a "lossy" process in terms of audio quality. Shifting a sample's pitch up or down by more than a few semitones can introduce undesirable artifacts.⁸

- **Technical Underpinnings:** Digital pitch-shifting works by resampling the audio data. When pitching down, the engine effectively plays the samples back slower, which can cause a loss of high-frequency information and introduce aliasing if the original sample rate is too low. When pitching up, the engine must interpolate

new samples, which can result in phasing issues or a thin, unnatural timbre often called the "chipmunk effect".⁶

- **Acceptable Range:** For a harmonically complex instrument like a piano, the perceptual limit for high-quality pitch-shifting is generally considered to be around **±2 to 3 semitones** (a major second to a minor third).⁹ Pushing beyond this range, especially on upward shifts, often yields audibly degraded results that can undermine the perceived quality of the instrument. This practical limit is the primary justification for a sampling strategy of one note per minor third.
- **Mitigation Strategies:** The quality of pitch-shifting can be improved by using higher-quality source material. Recording samples at a higher sample rate, such as 96 kHz, provides more data for the resampling algorithm to work with, resulting in cleaner down-pitching.⁶ For example, a 96 kHz sample pitched down by an octave (half speed) results in a 48 kHz audio stream, which is still professional quality. However, this doubles the size of the source assets, creating a new trade-off.

1.2.2 Simulating Dynamics with Velocity Layers

A real piano does not simply get louder when a key is struck harder; its entire timbral character changes, becoming brighter and richer in harmonics. This is simulated using velocity layers.

- **Concept:** Velocity layering involves recording the same note at multiple different dynamic levels, from very soft (*pianissimo*) to very loud (*fortissimo*). The game engine then selects which sample to play based on a mapped input velocity (e.g., how "hard" the player taps a tile).¹¹
- **Mobile Trade-off:** Each velocity layer is a full set of samples, so this technique multiplies the total asset count and memory footprint. While professional desktop piano libraries may use 12, 16, or even more layers¹³, this is not feasible on mobile. A practical target for a mobile piano is **2 to 4 velocity layers**. This provides a tangible sense of dynamic control without causing excessive asset bloat. For example, a 50-note set with 4 velocity layers requires 200 unique audio files. Some libraries attempt to "fake" additional layers by applying filters to a smaller set of samples, but the results are often unconvincing and can sound like they still only have the original number of layers.¹¹

1.2.3 Avoiding Repetition with Round-Robin Sampling

When the same note is triggered repeatedly in rapid succession, playing the exact same digital sample each time creates a robotic, mechanical sound often called the "machine-gun effect." Round-robin (RR) sampling is the technique used to combat this.

- **Concept:** RR involves recording several different takes of the same note at the same velocity. When the note is triggered, the engine cycles through these different takes in sequence (e.g., Sample 1, Sample 2, Sample 3, Sample 1...).¹⁴ This introduces the subtle, natural variations inherent in any acoustic performance, making the instrument feel more alive and realistic.
- **Mobile Trade-off:** Like velocity layers, RR sampling multiplies the total sample count. A common and effective number of RR variations is between **2x and 4x** per velocity layer. For our example 50-note set with 4 velocity layers, adding 2x round-robins would result in a total of $50 \times 4 \times 2 = 400$ individual samples. While this increases the asset size, it is a critical technique for achieving realism, especially for percussive instruments like the piano.¹⁴

1.3 Codec and Format Analysis for Mobile Deployment

The final piece of the sample-set strategy is choosing the right audio format. The choice of codec and its import settings in Unity directly impacts the final application size, the runtime memory usage, and the CPU overhead required for decoding.

- **PCM (in WAV/AIFF containers):** This is uncompressed audio data. It offers the highest possible fidelity and requires zero CPU cycles for decoding, as the data is already in a playable format. However, its file size is enormous, making it unsuitable for the final build package.¹⁶ Its primary role is in memory: when an AudioClip is set to "Decompress on Load" in Unity, the engine loads the compressed file from disk and stores it as raw PCM data in RAM.¹⁶ This is ideal for short sounds in a rhythm game, as it eliminates any potential for latency caused by runtime decompression.
- **Ogg Vorbis:** A high-quality, open-source lossy codec that generally provides

better compression than MP3 at a given bitrate.¹⁸ However, it is relatively CPU-intensive to decode.²⁰ Critically, for mobile targets, Unity will re-compress any imported Ogg Vorbis files into MP3 format anyway, which can introduce a slight quality degradation.¹⁶ Therefore, it offers little advantage over providing uncompressed source files and letting Unity handle the final encoding.

- **AAC (and MP3 on Mobile):** Advanced Audio Coding is the standard for Apple devices and is widely supported. In Unity, when targeting mobile platforms, compressed audio is typically encoded as MP3 to take advantage of hardware decoders.¹⁶ While efficient, MP3s created by some encoders can have a small amount of leading silence baked into the file, which is catastrophic for the precise timing required in a rhythm game.²² This risk is mitigated by importing source audio as uncompressed WAV/AIFF and letting Unity's own pipeline create the final MP3s.
- **ADPCM (Adaptive Differential Pulse Code Modulation):** A much older and simpler compression format with a fixed 4:1 compression ratio. While its quality is lower than modern codecs like Vorbis or AAC, its key advantage is its **extremely low CPU decoding cost**.²⁰ Because it is so computationally inexpensive, it is an excellent choice for short, frequently triggered sound effects where minimizing latency is the absolute top priority. For the hundreds of piano note samples that need to be played instantly, ADPCM represents a powerful and highly recommended option when using the "Compressed in Memory" load type.²⁵

Table 1.1: Audio Codec Comparison for Mobile Rhythm Game Notes

Feature	PCM (in RAM)	ADPCM	Ogg Vorbis	AAC / MP3
Typical Compression	None (1:1)	Fixed 4:1	High (e.g., 10:1)	High (e.g., 8:1)
Decoding CPU Cost	None	Very Low	High	Low (Hardware) / Medium (Software)
In-Memory Footprint	High	Low	Low	Low
Perceived Quality	Perfect	Good	Very Good	Very Good
Unity Load Type	Decompress on Load	Compressed in Memory	Compressed in Memory	Compressed in Memory

Rhythm Game Use Case	Excellent for latency , but highest RAM usage. Best if total sample memory is low.	Excellent balance of low CPU, low RAM, and good quality. A top contender for the piano samples.	High CPU cost makes it risky for high-polyphony, low-latency playback.	Good quality, but potential for encoder-introduced silence is a risk. ADPCM is often safer for this use case.
-----------------------------	---	--	--	---

Actionable Recommendations

- **Start with a 50-note sample set**, capturing one note every minor third across the central 5-6 octaves of the piano. This provides a strong balance between fidelity and asset size.
- **Implement 3-4 velocity layers** to provide meaningful dynamic expression. This is a critical feature for making classical music feel authentic.
- **Use at least 2x round-robin samples** for each velocity layer to eliminate the "machine-gun" effect on repeated notes.
- **Author all source audio as uncompressed WAV files** at 48 kHz, 16-bit, mono. This gives Unity the highest quality source material to work with.
- For the final build, set the piano note AudioClip import settings to **ADPCM compression** with the "**Compressed in Memory**" load type. This offers the best combination of low memory footprint and negligible decoding latency.
- If memory constraints are not severe, use the "**Decompress on Load**" setting. This will use more RAM but guarantees zero decoding latency by storing the samples as raw PCM data.
- **Prototype and listen.** The final decision on the sample-set density and compression settings must be made by listening to the results on target mid-range devices, not just by analyzing numbers.

Section 2: Architecture of a Low-Latency Real-Time Playback Engine

For a rhythm game, the software architecture responsible for triggering audio is not an auxiliary system; it is the core of the gameplay experience. Success is defined by

the ability to play a specific sound at a precise moment with minimal and, crucially, *consistent* latency. This section details the architectural choices, from Unity's built-in settings to the use of external middleware, and provides a blueprint for a high-performance C# note player.

2.1 Achieving Low-Latency Audio on Mobile: Unity's Core Systems and Settings

Before considering external tools, it is essential to configure Unity's native audio system for the lowest possible latency.

- **The DSP Buffer:** The Digital Signal Processing (DSP) buffer is the block of audio data that Unity's mixer processes at one time. The size of this buffer is the single most important setting for latency. A smaller buffer reduces the delay between a Play() call and the sound being heard but increases the risk of audio stutter or crackling (known as a buffer underrun) if the CPU cannot process and deliver the next block of audio in time.²⁷ For a rhythm game, this trade-off must lean heavily towards low latency. In Project Settings > Audio, the DSP Buffer Size must be set to **Best Latency**.²⁹
- **The Android Latency Problem:** A significant body of evidence points to a systemic issue with audio latency on the Android platform. Due to extreme hardware and OS fragmentation, many devices introduce significant additional latency through their audio drivers and signal processing pipelines.³¹ This can result in delays of 200-300 ms on some devices, a value that is immediately noticeable and renders a rhythm game unplayable.³² While Unity has made improvements since version 2019.1 by more aggressively using Android's low-latency OpenSL ES audio path, this is not a guaranteed fix across all devices.³³ This platform-specific variance represents a major project risk. iOS, by contrast, generally offers consistently low audio latency out of the box.³²
- **AudioClip Load Type:** As established in Section 1, the Load Type for the piano note samples is critical. To ensure instant playback without stutter, the audio data must be in memory before it is needed.
 - **Decompress on Load:** This setting uses the most RAM but guarantees zero latency from decompression at the moment of playback. The audio is fully uncompressed into memory when the clip is loaded.¹⁶
 - **Compressed in Memory:** This is a compromise, keeping the samples compressed in RAM to save space. Decompression happens on-the-fly when the sound is played. This adds a small amount of CPU overhead and latency.

Using a very fast codec like ADPCM makes this a viable option.²⁶

- **Streaming:** This setting is completely inappropriate for short, reactive sounds like piano notes, as it reads data from disk during playback and has the highest latency and CPU overhead.³⁴

2.2 Native Unity Audio vs. Middleware (FMOD/Wwise): A Comparative Analysis for Rhythm Games

Given the high risk of latency issues with Unity's native audio system, particularly on Android, a crucial architectural decision is whether to use it or to integrate a specialized audio middleware engine.

- **Unity Built-in Audio:**

- **Pros:** It is free, deeply integrated into the editor, and simple to use for basic audio needs. The workflow is straightforward, involving AudioSource components and AudioClip assets.³⁵
- **Cons:** The primary drawback is the high and unpredictable latency on Android, which is a potential project-killer.²² The system offers limited control over low-level scheduling, voice management, and hardware buffering. Profiling and debugging audio-specific issues can also be challenging compared to dedicated tools.²⁸

- **Middleware (FMOD & Wwise):**

- **Pros:** These are industry-standard, professional-grade audio engines that integrate with Unity. Their main advantage is that they largely bypass Unity's high-level audio mixer, communicating more directly with the platform's low-level audio APIs. This results in significantly **lower and more consistent latency** across all platforms, including Android.²³ They also provide vastly superior tools for mixing, authoring complex interactive audio events, and in-depth profiling and debugging.²⁰ Wwise is often regarded as more powerful and scalable for large AAA projects, while FMOD is frequently praised for its more intuitive, DAW-like user interface, which can be easier for smaller teams to adopt.³⁸
- **Cons:** Integrating middleware adds a new tool and workflow to the development process. There is a learning curve associated with both FMOD Studio and the Wwise Authoring tool.³⁷ Furthermore, there are licensing costs for commercial projects. Wwise, for example, has a free tier that is limited to 200 sound files, a limit that would be quickly exceeded by a multi-sampled

piano. FMOD's licensing is typically based on the project's development budget.³⁸

- **Native Plugins:** A third, more advanced option involves using a C# plugin to call the operating system's native audio libraries directly (e.g., OpenAL on iOS, AudioTrack or OpenSL ES via the NDK on Android).²⁹ This approach can yield the absolute lowest possible latency but requires significant platform-specific engineering expertise and is complex to implement and maintain.⁴⁰

The choice between these architectures is perhaps the most critical technical decision for the project. Relying solely on Unity's built-in audio carries a high risk of failure on a significant portion of the target Android market. Adopting middleware is a powerful mitigation strategy that outsources the complex problem of low-latency cross-platform audio to a specialized, proven solution.

Table 2.1: Audio Engine Architecture Comparison for Mobile Rhythm Game

Architecture	Latency Performance	Development Cost/Effort	Licensing Cost	Tooling & Workflow	Key Risks
Unity Built-in Audio	High & Variable (Android), Low (iOS). Inconsistent across devices.	Low. Integrated into Unity workflow.	None.	Simple, component-based. Limited profiling.	Unplayable latency on many Android devices, limiting market reach or causing poor reviews.
Middleware (FMOD/Wwise)	Low & Consistent on both iOS & Android. Bypasses Unity's high-latency paths.	Medium. Requires learning a separate authoring tool and API.	Varies. Free tiers with limitations (e.g., sound count), paid tiers for commercial release.	Superior. Advanced mixing, event authoring, and debugging tools.	Added workflow complexity. Budgeting for license fees.
Custom Native Plugin	Potentially the Lowest Possible, but	High. Requires deep, platform-spe	None (for self-developed).	None provided. Must be built from	High implementation complexity.

	implementati on-depende nt.	cific C++/Java/Ob j-C knowledge and wrapper code.		scratch.	Maintenance burden across OS updates. High risk of bugs.
--	-----------------------------------	--	--	----------	---

2.3 A High-Performance Pooled Note Player: C# Implementation Guide

Regardless of the underlying audio engine, the game logic must trigger notes with maximum efficiency. The sheer density of notes in classical piano music makes it impossible to instantiate a new `GameObject` or `AudioSource` for each note played. The performance overhead of `Instantiate()` calls and the subsequent pressure on the garbage collector would cause severe, gameplay-breaking stutters.⁴² The solution is to use an object pool.

An object pool pre-instantiates a collection of `AudioSource` components at startup. When a note needs to be played, an available `AudioSource` is taken from the pool, configured, and used. Once the sound finishes, the `AudioSource` is returned to the pool for reuse, avoiding any runtime allocation or destruction.⁴²

Below is C# pseudocode outlining a robust, pooled note player that supports the features discussed, such as pitch-shifting for unsampled notes and randomization for humanization. This implementation assumes the use of Unity's native audio system, but the same principles apply when triggering events in FMOD or Wwise.

C#

```
// C# Pseudocode for a High-Performance Pooled Note Player
```

```
using UnityEngine;
```

```
using System.Collections.Generic;
```

```
// A ScriptableObject to hold the data for a single piano note,
```

```
// including its different velocity layers and round-robin variations.
```

```
public class PianoNoteData : ScriptableObject
```

```

{
    public int midiNoteNumber;
    // Each element in the outer list is a velocity layer.
    // Each inner list contains round-robin samples for that layer.
    public List<AudioClip> velocityLayers; // Simplified for this example. A more robust solution
    would use a nested list or custom class.
}

```

```

public class PooledNotePlayer : MonoBehaviour
{

```

```

    public static PooledNotePlayer Instance;

```

```

    [Header("Pooling")]

```

```

    private GameObject audioSourcePrefab; // A prefab with just an AudioSource component.

```

```

    private int poolSize = 50; // Max polyphony.

```

```

    // A dictionary to map a MIDI note number to its closest sampled note data.

```

```

    private Dictionary<int, PianoNoteData> sampledNotes;

```

```

    [Header("Humanization")]

```

```

    private float pitchRandomness = 0.01f; // In semitones.

```

```

    private float volumeRandomness = 0.05f; // As a percentage.

```

```

    private Queue<AudioSource> readyPool;

```

```

    private int roundRobinIndex = 0; // Simple global RR for this example.

```

```

    void Awake()

```

```

    {

```

```

        Instance = this;

```

```

        InitializePool();

```

```

        // In a real project, load and populate the 'sampledNotes' dictionary here.

```

```

    }

```

```

    private void InitializePool()

```

```

    {

```

```

        readyPool = new Queue<AudioSource>();

```

```

        for (int i = 0; i < poolSize; i++)

```

```

        {

```

```
        var source = Instantiate(audioSourcePrefab,  
transform).GetComponent<AudioSource>();  
        source.gameObject.SetActive(false);  
        readyPool.Enqueue(source);  
    }  
}
```

```
public void PlayNote(int midiNote, float velocity) // Velocity 0.0 to 1.0  
{  
    if (readyPool.Count == 0)  
    {  
        // Voice stealing logic would go here. For now, we just log an error.  
        Debug.LogWarning("AudioSource pool exhausted! No available voices.");  
        return;  
    }  
}
```

```
    AudioSource source = readyPool.Dequeue();  
    source.gameObject.SetActive(true);
```

```
    // 1. Find the closest sampled note and calculate required pitch shift.  
    // (This logic would be more complex in a real implementation)  
    PianoNoteData noteData = FindClosestSampledNote(midiNote);  
    float pitchShift = CalculatePitchShift(midiNote, noteData.midiNoteNumber);
```

```
    // 2. Select the AudioClip based on velocity and round-robin.  
    // (This logic would select from different velocity layer lists)  
    AudioClip clipToPlay = SelectClip(noteData, velocity);
```

```
    // 3. Configure the AudioSource.  
    source.clip = clipToPlay;  
    source.pitch = Mathf.Pow(2f, pitchShift / 12f); // Convert semitone shift to pitch multiplier.
```

```
    // 4. Apply humanization.  
    source.pitch *= Random.Range(1f - pitchRandomness, 1f + pitchRandomness);  
    source.volume = velocity * Random.Range(1f - volumeRandomness, 1f +  
volumeRandomness);
```

```
    // 5. Play the sound.  
    source.Play();
```

```

        // 6. Schedule its return to the pool.
        StartCoroutine(ReturnToPoolAfterDelay(source, clipToPlay.length / source.pitch));
    }

    private AudioClip SelectClip(PianoNoteData noteData, float velocity)
    {
        // Placeholder: A real implementation would map velocity to different lists of clips.
        // It would also use a proper round-robin system per note.
        roundRobinIndex = (roundRobinIndex + 1) % noteData.velocityLayers.Count;
        return noteData.velocityLayers;
    }

    private PianoNoteData FindClosestSampledNote(int targetMidiNote)
    {
        // Placeholder: Logic to find the best PianoNoteData from the 'sampledNotes' dictionary.
        // This would find the key in the dictionary closest to 'targetMidiNote'.
        return sampledNotes; // Example: return C4
    }

    private float CalculatePitchShift(int targetNote, int sampledNote)
    {
        return targetNote - sampledNote; // Shift in semitones.
    }

    private System.Collections.IEnumerator ReturnToPoolAfterDelay(AudioSource source, float delay)
    {
        yield return new WaitForSeconds(delay);
        source.gameObject.SetActive(false);
        readyPool.Enqueue(source);
    }
}

```

Actionable Recommendations

- **Set DSP Buffer to "Best Latency"** in Project Settings > Audio as the first step.

- **Strongly consider using audio middleware (FMOD or Wwise).** The risk of unplayable latency on Android with Unity's native audio is too high for a commercial rhythm game. Prototype with a middleware solution early to validate its performance benefits.
 - **Implement a robust AudioSource object pool** from the very beginning of development. Do not use Instantiate() for note playback under any circumstances.
 - Design the pooling system to be the central point of control for all note playback, handling sample selection, pitch-shifting, and humanization logic.
 - For note AudioClips, use "**Decompress on Load**" if RAM permits, or "**Compressed in Memory**" with the **ADPCM** codec if RAM is a concern.
-

Section 3: Sourcing and Licensing the Classical Repertoire

The heart of a classical music rhythm game is its repertoire. Selecting the pieces, understanding their technical demands, and, most importantly, navigating the legal complexities of their use are foundational tasks. This section provides a guide to analyzing the musical requirements of the classical canon and sourcing the necessary data in a legally sound manner for commercial use.

3.1 Typical Pitch Range Demanded by Cornerstone Classical Pieces

The choice of musical repertoire directly influences the required size of the piano sample set. Music from different eras was composed for instruments with different keyboard ranges.

- **Baroque (e.g., J.S. Bach):** Composers like Bach wrote for the harpsichord and clavichord, which typically had a range of four to five octaves. A piece like Bach's *Invention No. 1 in C Major, BWV 772* stays comfortably within a two-octave range centered around middle C, making it very easy to cover with a small sample set.⁴⁵
- **Classical (e.g., W.A. Mozart):** The fortepianos of Mozart's time expanded the keyboard to around five or five-and-a-half octaves. Analysis of his works shows that his highest written note is F#6 (or Gb6, MIDI note 90), providing a clear upper boundary for his repertoire.⁴⁷
- **Romantic (e.g., F. Chopin, F. Liszt):** The Romantic era saw the piano evolve into

the powerful, wide-ranging instrument we know today. Chopin's works, such as the *Fantaisie-Improvisation*, regularly utilize a range of 6.5 octaves or more, demanding a much larger sample set to be rendered authentically.⁴⁸ Composers like Liszt pushed the instrument to its absolute limits, frequently employing the entire 88-key range of the modern concert grand.

To accommodate a diverse and popular classical repertoire, a sample set covering at least **six octaves (approximately 73 keys)** is highly advisable. This ensures that most works by Bach, Mozart, and a significant portion of Chopin can be played with minimal pitch-shifting. A full 88-key set is only strictly necessary for the most virtuosic late-Romantic and 20th-century pieces.

3.2 Navigating the Public Domain: Licensing for Scores, MIDI, and Recordings

Using "public domain" music is not as simple as finding a file online and putting it in a game. For any given piece of music, up to three distinct copyrights may exist, and each must be respected.

1. **Copyright of the Composition:** This protects the original melody and harmony created by the composer. For composers like Bach (d. 1750), Mozart (d. 1791), and Chopin (d. 1849), their compositions are firmly in the public domain worldwide, meaning the underlying work can be used freely by anyone for any purpose.⁵⁰
2. **Copyright of the Edition (The Score):** A modern publisher's version of a public domain work (e.g., a 2024 edition from Bärenreiter) includes new scholarly work, typesetting, and fingerings, and is protected by its own copyright. To be legally safe, one must use a scan of an *old edition* that is itself in the public domain. In the United States, a general rule is that any work published before 1929 is in the public domain.⁵¹
3. **Copyright of the Performance (The Recording/MIDI):** A musician's performance of a piece is a separate creative work and is protected by copyright. This means one cannot use a commercial recording without a license. Crucially, this extends to **MIDI files**. A MIDI file created by a human is a form of recorded performance. Therefore, one cannot simply download a random MIDI file of a Chopin waltz and use it in a commercial game, as that MIDI file is likely the copyrighted work of the person who sequenced it.⁵⁰

The safest legal strategy for a commercial project is to **control the creation of the**

performance data. This involves:

1. Sourcing a score from an edition that is verifiably in the public domain (e.g., from IMSLP).
2. Using that score to create a new, original MIDI or MusicXML file. This new file is a work owned by the developer and can be used without restriction.

3.3 Recommended Data Sources: Public Domain MIDI & MusicXML Libraries

While creating MIDI from scratch is the safest path, several online resources can provide public domain scores and, in some cases, permissively licensed MIDI files.

- **International Music Score Library Project (IMSLP):** This is the definitive resource for obtaining scans of public domain sheet music. IMSLP has a rigorous policy of verifying the copyright status of its scores in major legal jurisdictions (Canada, US, EU).⁵² This should be the primary source for the scores used to create game charts.
- **Mutopia Project:** An excellent resource that provides over 2,000 pieces of classical music. All content is either in the public domain or distributed under permissive Creative Commons licenses. It offers files in PDF, LilyPond, and, most importantly, **MIDI format**, making it a strong candidate for sourcing game-ready data.⁵⁶
- **MuseScore.com:** This resource must be approached with extreme caution. MuseScore.com is a community site that hosts a mixture of user-created arrangements of copyrighted pop songs (which are illegal to use commercially without a separate license) and arrangements of public domain works.⁵⁷ While the MuseScore *software* is free to use for commercial purposes⁵⁹, the content on the *website* is not. One must carefully filter for public domain works and scrutinize the license of each individual user submission. Relying on this as a primary source for a commercial game is legally risky.

Table 3.1: Public Domain Music Data Source Comparison

Library Name	Primary Content	Data Formats	Licensing Model	Commercial Use Recommendation
--------------	-----------------	--------------	-----------------	-------------------------------

IMSLP	Sheet Music (Scans)	PDF	Public Domain / CC	Highly Recommended. The gold standard for sourcing legally safe scores from which to create your own MIDI/MusicXML files.
Mutopia Project	Scores & MIDI	PDF, MIDI, LilyPond	Public Domain / CC	Highly Recommended. A reliable source for permissively licensed MIDI files that can be used directly.
Kunstderfuge	Scores & MIDI	PDF, MIDI	Public Domain / Custom	Recommended with Verification. Good source, but verify the license terms for each piece before use.
MuseScore.com	User-Generated Scores	MusicXML, MIDI, PDF	Mixed (PD, Copyrighted, CC)	Not Recommended for Commercial Use. High risk of using improperly licensed content. Legal status of user-created MIDI of PD works is ambiguous.

Actionable Recommendations

- **Define the target repertoire early** to inform the required range of the piano

sample set. Aim for at least a 6-octave range to cover popular works.

- **Adopt a strict licensing policy.** The safest approach is to create all game chart data (MIDI/MusicXML) in-house.
 - **Use IMSLP as the primary source for sheet music.** Download scans of editions published before 1929 to ensure they are in the public domain in the US.
 - **Use the Mutopia Project as a source for reliable, permissively licensed MIDI files** if creating them from scratch is not feasible.
 - **Avoid using MIDI files from aggregator sites or MuseScore.com** for commercial purposes unless an explicit, commercially permissive license (like CCO or CC BY) is attached and can be verified.
 - When in doubt, **consult a lawyer specializing in intellectual property** to vet the content licensing strategy before release.
-

Section 4: Implementing a Data-Driven Chart Pipeline

A successful rhythm game is built on a robust and efficient content pipeline. This pipeline is the process that transforms raw musical information, such as a score or MIDI file, into a structured data format that the game engine can use to generate gameplay. This section outlines the steps to build such a pipeline, from parsing source files to designing a game-ready JSON format and automating the workflow.

4.1 From Score to Game Data: Parsing MusicXML and MIDI in C#

The first step is to choose a source format for the musical data and a library to parse it.

- **MIDI vs. MusicXML:**
 - **MIDI (Musical Instrument Digital Interface):** This is a performance-based format. It contains a stream of events like "note on," "note off," "velocity," and timing information. It does not contain rich notational details like key signatures or slurs, but for a piano-tiles game, its direct event-based nature is often sufficient and simpler to parse.
 - **MusicXML:** This is a descriptive, score-based format. It is an XML-based standard that represents the full musical score, including notes, rests,

measures, dynamics, and articulations. It is more verbose and complex than MIDI but provides a more complete musical picture.⁵⁶ For this project's needs, MIDI is likely the more direct and efficient source format.

- **C# Parsing Libraries:**

- **For MusicXML:** As an XML-based format, MusicXML can be parsed using .NET's built-in System.Xml libraries. However, for a more robust and schema-aware solution, a dedicated library is recommended. MusicXml.Net is a NuGet package designed for this purpose, offering a straightforward API to access score elements.⁶¹ For performance-critical parsing, a low-level, allocation-free parser like U8XmlParser could also be considered.⁶³
- **For MIDI:** Several open-source C# libraries for MIDI parsing exist, though a popular, universally maintained standard is less clear. Projects like Tonejs/MidiConvert (JavaScript) provide an excellent reference for the data structure of a parsed MIDI file, which can be replicated in C#. ⁶⁴ Given the relative simplicity of the MIDI format, writing a custom parser that extracts the necessary note-on/off events, velocity, and timing is a feasible task.

4.2 Designing a Robust JSON Chart Format for Rhythm Gameplay

Once parsed, the musical data must be stored in a format optimized for the game engine. JSON (JavaScript Object Notation) is an ideal choice due to its light weight, human readability, and ease of parsing in Unity (either with the built-in JsonUtility or a more powerful library like Newtonsoft.Json).⁶⁵

A well-designed chart format should be self-contained and include all necessary information for both gameplay and display.

- **Proposed JSON Structure:**

```
JSON
{
  "songID": "bach_invention_1",
  "title": "Invention No. 1 in C Major",
  "composer": "J.S. Bach",
  "initialBPM": 120.0,
  "timeSignature": {
    "numerator": 4,
```

```

    "denominator": 4
  },
  "totalDurationSeconds": 88.5,
  "difficulty": "Easy",
  "tempoMap": [
    { "time": 0.0, "bpm": 120.0 },
    { "time": 82.0, "bpm": 110.0 } // Example: ritardando at the end
  ],
  "notes":
}

```

- **Key Fields Explained:**

- **Metadata (songID, title, composer, etc.):** Used for UI display and song selection.
- **tempoMap:** This is critical. Classical music is rarely played at a perfectly constant tempo. A tempo map is an array of points in time where the BPM changes, allowing for expressive accelerando and ritardando.⁶⁶ The game's scroll speed logic must read from this map to stay in sync.
- **notes Array:** This is the core of the chart. Each object represents a single tile in the game. Storing time in seconds provides a universal, easy-to-use value for spawning tiles, independent of BPM changes. midiNote and velocity are passed directly to the audio engine, while duration and lane are used by the gameplay and visual systems.

4.3 Automation: Tools and Scripts for Chart Creation and Slicing

Manually authoring JSON files is not a scalable workflow. The quality and quantity of content will depend on the quality of the internal tools built to support the process. The pipeline should be viewed as a full level-design workflow, not just a data conversion step.

- **The Chart Editor:** A custom Unity Editor window is an essential investment. This tool should provide a visual interface for "charting" a song.⁶⁶ Its key features should include:
 1. **Import:** Load a source MIDI or MusicXML file.
 2. **Parse & Display:** Parse the file and display the notes on a piano roll-style timeline against an audio waveform of a reference recording.

3. **Charting Tools:** Allow a designer to select, delete, or modify notes. The most important function will be to assign notes to one of the game's visual lanes (e.g., by clicking or dragging). This is a creative process of translating a complex piano part into a playable 4-lane game.
 4. **Tempo Editing:** Provide an interface to create and edit the tempoMap.
 5. **Export:** Save the final chart data to the structured JSON format.
- **Slicing into "Time Packages":** For long and dense classical pieces, loading the entire notes array into memory at the start of a level is inefficient and can cause memory spikes. A more advanced pipeline would include a pre-processing step or runtime logic to slice the note data into time-based chunks. For example, the game could load the notes for the first 15 seconds, and while playing, asynchronously load the notes for the next 15-second block in the background. This keeps the active memory footprint low and consistent, regardless of song length.

Actionable Recommendations

- **Choose MIDI as the primary source format.** Its event-based structure is a more direct match for the game's needs than the descriptive structure of MusicXML.
- **Invest time in building a custom C# MIDI parser** or thoroughly vetting an existing open-source library. The parser should accurately extract note-on/off events, velocity, and timing ticks.
- **Design the JSON chart format to be self-contained and extensible.** Include a tempoMap from the beginning to support expressive timing.
- **Prioritize the development of a custom Unity chart editor.** This tool is critical for efficient and creative level design. A good editor will dramatically increase the quality and quantity of content that can be produced.
- **Plan for content slicing.** For songs longer than ~2 minutes, implement a system to load note data in time-based chunks to manage memory usage effectively.

Section 5: Mobile Performance, Memory, and Polyphony Management

Achieving a stable 60 fps on mid-range mobile devices while running a

high-polyphony, low-latency audio engine is a significant engineering challenge. This section addresses the key performance bottlenecks—CPU, memory, and voice count—and outlines strategies for profiling, optimizing, and managing these resources under the intense pressure of rhythm gameplay.

5.1 Performance Benchmarks: Deconstructing Existing Rhythm Game Audio Engines

Analysis of popular existing games like *Piano Tiles* reveals a much simpler audio model than the one proposed for this project. These games often do not synthesize music from individual samples in real time with full polyphony. Instead, they frequently use a pre-recorded backing track and the player's taps trigger pre-sequenced, single-note samples, or simply unmute the corresponding note in the mix.⁶⁷ This approach is far less demanding. A game that aims to render complex classical music polyphonically from individual samples is operating in a different performance class and must be architected accordingly.

General Mobile Audio Performance Targets:

- **CPU Usage:** Audio processing should be kept under a strict budget. A common industry guideline is to keep the audio thread's CPU usage **below 20%** of a single core's capacity. This leaves sufficient headroom for game logic, rendering, and UI, and helps prevent thermal throttling on the device.¹⁸
- **Memory Usage:** Audio assets can be a dominant consumer of both storage and RAM. A practical goal is to keep the total size of all audio assets to **15-20% of the final application size**.¹⁸ This requires aggressive compression and smart memory management.
- **Profiling Tools:** Continuous profiling is non-negotiable.
 - **Unity Profiler:** The built-in Audio Profiler is the first line of defense. It provides detailed frame-by-frame information on playing sources, paused sources, total real and virtual voices, memory consumption, and the CPU cost of the audio thread.²⁶
 - **Native Tools:** For deeper, device-specific insights, Xcode Instruments for iOS and Android Studio Profiler for Android are essential.⁷⁰ These tools can reveal low-level performance issues that the Unity Profiler might not expose.

5.2 Polyphony Management Under Pressure: Voice Stealing and Capping

Polyphony refers to the number of notes playing simultaneously. In piano music, with the use of the sustain pedal and rapid passages, the number of active voices can quickly skyrocket. Mobile devices and game engines have a hard limit on the number of "real" voices they can play at once. Exceeding this limit results in notes being abruptly cut off, which is highly undesirable.⁷¹

- **Voice Limits in Unity:** Unity's audio settings allow for configuring the **Max Real Voices** and **Max Virtual Voices**. By default, the real voice limit is 32. While this can be increased up to 255, it is advisable to keep it as low as possible, as each voice consumes CPU and memory. A limit between **32 and 64** is a realistic target for mobile.²⁶
- **Voice Stealing:** When the real voice limit is reached, the engine must "steal" a voice from a currently playing note to play a new one. This is where a voice management or virtualization system comes in.
 - **Unity's Default System:** When the real voice limit is hit, Unity will convert the lowest-priority sound into a "virtual voice." A virtual voice continues to track its playback position but produces no sound and consumes almost no CPU. If a real voice becomes free later, the virtual voice can be promoted back to a real voice and resume playing from the correct time. The priority is determined first by the `AudioSource.priority` property (0 is highest), and for sources with the same priority, by audibility (volume and distance).²⁶
 - **Custom Musically-Aware Algorithm:** For a piano, Unity's default system may not be ideal, as it has no musical context. A more sophisticated, custom voice-stealing algorithm implemented within the `PooledNotePlayer` can make more intelligent decisions. When the pool is empty and a new note must be played, the algorithm should choose a voice to steal based on musical heuristics:
 1. **Prioritize by Age (LRU):** First, consider stealing the voice that has been playing the longest (Least Recently Used).⁷²
 2. **Prioritize by Amplitude:** Next, consider stealing the voice with the lowest current volume. A note that has already decayed significantly is less noticeable when cut off.
 3. **Prioritize by Release Phase:** A note whose key has already been "released" (i.e., it's in its sustain-pedal decay phase) is a prime candidate for stealing.
 4. **Prioritize by Musical Role:** One could even implement logic to protect

foundational notes (e.g., low bass notes) from being stolen by higher, more decorative melodic notes.

5.3 Profiling and Optimizing Audio Memory and CPU Usage on Target Devices

Optimization is an iterative process of measuring, identifying bottlenecks, and applying targeted fixes.

- **Using the Unity Profiler:**
 - In the **CPU Usage** profiler, pay close attention to markers related to `Audio.Play`, `Audio.Update`, and, if using middleware, `FMOD.Update` or `Wwise.Update`. Spikes in these areas indicate performance issues.
 - In the **Audio** profiler pane, monitor the "Playing Sources" and "Voices" counts. If the voice count consistently hits the "Max Real Voices" limit, it's a sign that the voice management system is under pressure and may be audibly culling sounds.²⁶
- **Memory Optimization Strategies:**
 - **Preload Audio Data:** This setting should be disabled for any audio that is not needed the instant a scene loads. For a large library of songs, the piano samples can be preloaded, but the audio for specific song charts should only be loaded when that song is selected. This can be managed manually with `AudioClip.LoadAudioData()` and `AudioClip.UnloadAudioData()`.²⁶
 - **Force to Mono:** Piano samples should be authored as mono files. This halves their memory footprint compared to stereo with no loss of quality for a non-spatialized instrument.²⁶
 - **Compression and Load Type:** This is the most significant memory optimization. As determined in Section 1, using Compressed in Memory with the ADPCM codec provides an excellent balance of low RAM usage and low CPU overhead.³⁴
- **CPU Optimization Strategies:**
 - **Disable Unused Sources:** An `AudioSource` component that is muted but still enabled continues to consume some CPU resources for updates. It is more efficient to disable the entire `AudioSource` component or the `GameObject` it is attached to.²⁶ The object pooling system naturally handles this by deactivating the `GameObject`.
 - **Architectural Choice:** The most impactful CPU optimization is the choice of

audio engine. Middleware or a native plugin will almost always offer better CPU performance for high-polyphony scenarios than Unity's built-in audio system.

Actionable Recommendations

- **Establish a strict performance budget early:** Target <20% CPU for audio and <20% of total app size for audio assets.
 - **Profile continuously on a range of target devices,** including low-end models from 2019-2020. Do not rely solely on profiling in the Editor.
 - **Set Unity's Max Real Voices to a reasonable limit for mobile, such as 48 or 64,** and monitor the "Voices" count in the profiler to see if this limit is being hit.
 - **Implement a custom, musically-aware voice-stealing algorithm** within the PooledNotePlayer. The algorithm should prioritize stealing the oldest, quietest, and least musically important notes first.
 - **Use the Unity Profiler to hunt down memory and CPU spikes.** Disable Preload Audio Data for non-essential clips and ensure all piano samples are mono and efficiently compressed.
-

Section 6: Advanced Enhancement Techniques for Expressivity

Once the core playback engine is performant and reliable, the focus can shift to enhancing the sonic quality and expressivity of the piano. Basic sample playback can sound static and lifeless; the key to a convincing performance lies in simulating the subtle nuances of an acoustic instrument. This section explores several advanced techniques and their performance implications on mobile.

6.1 Creating Ambience: The Trade-offs Between Runtime Convolution and Baked Reverb

Reverberation (reverb) is essential for placing the piano in a believable acoustic space

and giving the sound warmth and depth. There are three primary methods for applying reverb, each with a different trade-off between realism and performance cost.

- **Baked Reverb:** This involves including the natural reverb tail of the recording space directly within each piano sample.
 - **Pros:** It has **zero additional CPU cost** at runtime, as the reverb is simply part of the pre-rendered audio file.
 - **Cons:** It is completely inflexible. The reverb character is fixed and cannot be changed. More importantly, when multiple notes with long baked-in tails are played, the overlapping reverbs can accumulate and create a "muddy," washed-out sound.
- **Algorithmic Reverb:** This method uses complex mathematical algorithms (networks of delays, filters, and feedback loops) to simulate the reflections of a room. Unity's built-in AudioReverbFilter is an example of an algorithmic reverb.
 - **Pros:** It is highly flexible and controllable, allowing for real-time adjustment of parameters like decay time, room size, and diffusion. It is also relatively **low in CPU demand**, making it a very suitable choice for mobile platforms.⁷³
 - **Cons:** It can sometimes sound less realistic or "natural" than convolution reverb, as it is a simulation rather than a snapshot of a real space.
- **Convolution Reverb:** This technique uses an "impulse response" (IR)—an audio recording made in a real acoustic space (like a concert hall or church)—to process the dry audio signal. It effectively "places" the sound in that exact space.
 - **Pros:** It offers **unparalleled realism and authenticity**. This is the go-to method for film scoring and professional audio production when a lifelike environment is required.⁷³
 - **Cons:** It is **extremely CPU-intensive**. The mathematical convolution operation is computationally expensive, especially with long, high-quality IRs. Running real-time convolution reverb on mobile is generally not viable for a high-polyphony instrument, as it would quickly consume the entire CPU budget.⁷³

For this project, the most pragmatic and effective approach is to use **dry piano samples** (with very little natural room sound) and send their output to a single, high-quality **algorithmic reverb** placed on an Audio Mixer group. This provides a good balance of quality, control, and performance, as the reverb calculation only needs to be done once per frame for the mixed signal, rather than on every individual voice.

6.2 Velocity-Layering and Dynamic EQ for Expressiveness

Simulating the way a piano's tone changes with playing intensity is key to expressivity.

- **Velocity-Layer Crossfading:** Instead of abruptly switching between velocity-layer samples, a much smoother and more realistic transition can be achieved by crossfading between two adjacent layers. For example, if a note is played with a MIDI velocity of 70 (where 64 is the boundary between a *mezzo-forte* and *forte* layer), the engine can play both the *mf* and *f* samples simultaneously, with the volume of the *mf* sample at 80% and the *f* sample at 20%. This technique eliminates the noticeable timbral jumps of simple layer switching but comes at the cost of doubling the number of voices required for any note that falls between layers. This places even more pressure on the polyphony management system.
- **Dynamic EQ:** A real piano's sound becomes significantly brighter (i.e., richer in high-frequency harmonics) when played forcefully. This effect can be simulated using a dynamic equalizer (EQ). A dynamic EQ is an equalizer whose parameters (like gain) are controlled by the amplitude of the input signal.⁷⁵ In this context, the note's velocity can be used to control an EQ filter.
 - **Unity Implementation:** This can be achieved by adding an `AudioParametricEQFilter` component to the `AudioSource` prefabs in the object pool. The script in the `PooledNotePlayer` would then adjust the `frequencyGain` property of a high-frequency band based on the incoming velocity value.⁷⁶ A low-velocity note would receive little to no gain, while a high-velocity note would receive a significant boost in the high frequencies, simulating the bright, sharp attack of a hard-struck key. This technique adds a per-voice DSP cost and must be profiled carefully, as running dozens of simultaneous EQ filters can be CPU-intensive.⁷⁷

6.3 The Human Touch: Applying Micro-Randomization for Realism

Even with round-robins, a digitally perfect performance can sound sterile. Introducing tiny, controlled imperfections—micro-randomization—is a powerful way to add a final layer of humanization and realism.

- **Pitch Randomization:** A real acoustic piano is never perfectly in tune. Each string has minute variations. This can be simulated by applying a very small, random pitch shift to each note as it is played. A range of **±1 to 3 cents** (where 100 cents = 1 semitone) is often sufficient to break up the digital perfection without sounding out of tune. This is easily implemented by scripting the `AudioSource.pitch` property.⁷⁸
- **Volume Randomization:** Similarly, a subtle random variation in volume (e.g., **±0.5 dB** or a multiplier of `Random.Range(0.95f, 1.05f)`) can help to avoid mechanical-sounding repeated notes.
- **Timing Randomization:** For the highest level of humanization, a very small random offset (e.g., **±5-10 milliseconds**) can be applied to the scheduled playback time of each note. This mimics the natural, subtle timing fluctuations of a human performer. This must be implemented with extreme care, as too much randomization can make the gameplay feel sloppy or unresponsive.

The implementation of these enhancement techniques must be approached with performance in mind. Each one adds a small amount of per-voice CPU overhead. When multiplied by a high polyphony count, this overhead can become significant. A critical design consideration should be a **quality scaling system**, where these enhancements can be dynamically enabled or disabled based on the device's capabilities, ensuring the game runs smoothly on low-end hardware while providing the richest possible experience on high-end devices.

Actionable Recommendations

- **Use dry piano samples and a single, high-quality algorithmic reverb** on an Audio Mixer bus for ambience. Avoid real-time convolution reverb.
- **Implement velocity layer crossfading** for smooth dynamic transitions, but be aware that this will double the voice count for many notes and require a robust voice-stealing system.
- **Experiment with dynamic EQ** by scripting Unity's `AudioParametricEQFilter` to boost high frequencies on high-velocity notes. Profile this feature carefully due to its per-voice CPU cost.
- **Apply subtle micro-randomization** to the pitch (**±2 cents**) and volume (**±0.5 dB**) of every note played to add a final layer of realism.
- **Design a graphics/audio quality settings menu** that allows these CPU-intensive enhancement features to be disabled on lower-end devices to

maintain a stable 60 fps.

Section 7: Project Implementation Roadmap and Risk Mitigation

Developing a technically ambitious game like this requires a structured approach and a proactive strategy for managing risks. This section provides a phased implementation roadmap to guide the project from an initial prototype to a polished release, followed by an analysis of the most significant technical and legal risks and their corresponding mitigation strategies.

7.1 A Phased Implementation Roadmap: From Prototype to Production

A phased approach allows for early validation of core mechanics and systematic de-risking of the project's most challenging aspects.

- **Phase 1: Core Audio Prototype & Architecture Validation (Weeks 1-3)**
 - **Objective:** To validate the core real-time sample playback mechanic and, most importantly, to make an informed, evidence-based decision on the audio architecture.
 - **Key Tasks:**
 1. Acquire a small, temporary piano sample set (e.g., one octave of single-velocity samples).
 2. Implement a basic version of the PooledNotePlayer (Section 2.3).
 3. Create a simple test scene that can play notes from a hardcoded array or a basic JSON file.
 4. **Crucially, build two parallel prototypes:** one using Unity's native audio system (with DSP Buffer set to Best Latency) and one using a middleware solution (FMOD is often recommended for its easier learning curve for indie developers ³⁷).
 5. Deploy both prototypes to a range of target Android and iOS devices (especially low- to mid-range Android phones) and measure the touch-to-sound latency. This empirical data will determine the architectural path forward.
- **Phase 2: Content Pipeline & Charting Tools (Weeks 4-7)**

- **Objective:** To build the tools and processes necessary to create game content efficiently.
- **Key Tasks:**
 1. Based on the Phase 1 decision, finalize the audio engine integration.
 2. Finalize the sample-set strategy (sample count, velocity layers, round-robins) and begin sourcing or recording the final piano samples.
 3. Source and legally clear the initial set of public domain scores (Section 3).
 4. Develop the first version of the custom Unity chart editor for converting MIDI files into the game's JSON format (Section 4.3).
 5. Use the editor to create full charts for 5-10 initial songs.
- **Phase 3: Gameplay & UI Implementation (Weeks 8-12)**
 - **Objective:** To build the core gameplay loop and user interface around the established audio and content pipelines.
 - **Key Tasks:**
 1. Implement the visual "falling tiles" mechanic, ensuring it is perfectly synchronized with the note data from the JSON charts.
 2. Develop the player input system, scoring logic, and combo mechanics.
 3. Build the main UI screens: song selection, settings, results screen, etc.
 4. Integrate the PooledNotePlayer so that player taps trigger the correct notes.
- **Phase 4: Polishing, Optimization & Enhancement (Weeks 13-16)**
 - **Objective:** To refine the gameplay experience, add expressive audio features, and optimize for release.
 - **Key Tasks:**
 1. Implement the advanced audio enhancement techniques (reverb, dynamic EQ, randomization) with a quality scaling system (Section 6).
 2. Implement a robust, musically-aware voice-stealing algorithm (Section 5.2).
 3. Conduct extensive performance profiling on all target devices and optimize CPU, GPU, and memory usage.
 4. Refine game feel, visual effects, and user feedback.
- **Phase 5: Pre-Launch & Release**
 - **Objective:** To prepare the game for submission to the app stores.
 - **Key Tasks:**
 1. Conduct closed beta testing to gather feedback and identify bugs.
 2. Integrate any necessary SDKs (analytics, ads, etc.).
 3. Create all required store assets (icons, screenshots, promotional text).
 4. Submit the final build to the Google Play Store and Apple App Store.

7.2 Identifying and Mitigating Key Technical and Legal Risks

Proactively identifying and planning for major risks is essential to prevent project failure.

Table 7.1: Key Technical and Legal Risk Mitigation Strategies

Risk Area	Description of Risk	Probability / Impact	Mitigation Strategy	Key Sources
Audio Latency	The game is unplayable on a significant portion of the Android market due to high and variable audio latency in Unity's native audio pipeline.	High / Critical	1. Prototype Early: Test latency with both native Unity audio and a middleware solution (FMOD/Wwise) on target devices in the first weeks of development. 2. Adopt Middleware: The strongest mitigation is to use middleware, which provides a proven, low-latency audio path. 3. Limit Market: If using native audio, consider only officially supporting devices with the android.hardware.audio.pro flag, but this severely limits reach.	32

Asset Size & Memory	The final application size, bloated by a large piano sample library, exceeds store limits (e.g., 200 MB initial download on Google Play) or the RAM of low-end devices, leading to poor download rates or crashes.	High / High	1. Budgeting: Establish a strict storage and RAM budget from the project's outset. 2. Efficient Codecs: Use a highly efficient codec like ADPCM for the note samples. 3. Asset Delivery: Utilize Play Asset Delivery (Android) and On-Demand Resources (iOS) to package songs as optional, downloadable content. This keeps the base game small and allows for a massive repertoire.	80
Legal & Licensing	The game is subject to a copyright claim or removal from stores due to the improper use of music assumed to be in the public domain.	Medium / Critical	1. Verify Everything: Do not assume any score or MIDI file is free for commercial use. Verify the copyright status of the composition, the score edition, and the performance/MIDI file separately. 2. Create In-House: The safest legal path	50

			is to create original MIDI/chart files from scores that are verifiably in the public domain (e.g., editions published pre-1929 from IMSLP). 3. Consult Counsel: For a commercial release, engage a lawyer specializing in intellectual property to review the content licensing strategy.	
Performance Bottlenecks	The combination of high polyphony and advanced DSP effects (reverb, EQ) overwhelms the CPU on mid-range devices, causing the frame rate to drop below 60 fps and the audio to stutter.	High / High	1. Continuous Profiling: Profile CPU and memory usage on target devices throughout development, not just at the end. 2. Quality Scaling: Design and implement a quality settings system that can disable or reduce the complexity of CPU-intensive features (e.g., dynamic EQ, reverb, crossfading) on lower-end	26

			hardware. 3. Robust Voice Management: Implement an intelligent voice-stealing algorithm to gracefully handle high polyphony without audible artifacts.	
--	--	--	--	--

Actionable Recommendations

- **Prioritize the Phase 1 prototype.** The decision between native Unity audio and middleware is the most critical one in the project and must be based on empirical data from target devices.
- **Treat the content pipeline as a first-class feature.** The quality of the chart editor will directly impact the quality and quantity of the game's content.
- **Plan for a downloadable content (DLC) model from the start.** Use Play Asset Delivery and On-Demand Resources to manage the game's size and allow for future expansion of the music library.
- **Maintain a risk register.** Document the key risks identified here and review them regularly throughout the project lifecycle, adjusting mitigation strategies as needed.

Alıntılanan çalışmalar

1. Best hardware sampler for sampling a piano? - Gearspace, erişim tarihi Haziran 30, 2025, <https://gearspace.com/board/electronic-music-instruments-and-electronic-music-production/1067024-best-hardware-sampler-sampling-piano.html>
2. Audio File Size Calculator – Colin Crawley, erişim tarihi Haziran 30, 2025, <https://www.colincrawley.com/audio-file-size-calculator/>
3. Audio Recording Calculator - Sound Devices, erişim tarihi Haziran 30, 2025, <https://www.sounddevices.com/audio-recording-calculator/>
4. Best Digital Pianos & Keyboards 2025 (All Price Points), erişim tarihi Haziran 30, 2025, <https://www.pianodreamers.com/best-digital-pianos-and-keyboards/>
5. Best Digital Piano & Keyboards 2025: I Tested 100 + Models, erişim tarihi Haziran

- 30, 2025, <https://keyboardkraze.io/best-digital-pianos-and-keyboards/>
6. How to Pitch Shift Without Losing Quality - Sage Audio, erişim tarihi Haziran 30, 2025, <https://www.sageaudio.com/articles/how-to-pitch-shift-without-losing-quality>
 7. How much does the sample rate of a vocal track affect the quality when it is pitch shifted?, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/AdvancedProduction/comments/9hbyzo/how_much_does_the_sample_rate_of_a_vocal_track/
 8. Final mix pitch shift - Gearspace, erişim tarihi Haziran 30, 2025, <https://gearspace.com/board/mastering-forum/366649-final-mix-pitch-shift.html>
 9. Pitch Shifting 101: Super Creative Tips & Advanced Techniques - Unison Audio, erişim tarihi Haziran 30, 2025, <https://unison.audio/pitch-shifting/>
 10. Does changing the pitch of a sample change the key? (Please don't make fun of me lol) : r/ableton - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/ableton/comments/195bv0d/does_changing_the_pitch_of_a_sample_change_the/
 11. The Steinway Experiment - Pianobook, erişim tarihi Haziran 30, 2025, <https://www.pianobook.co.uk/packs/the-steinway-experiment/>
 12. how many velocity layers do the pianos have? - Nord User Forum, erişim tarihi Haziran 30, 2025, <https://www.norduserforum.com/viewtopic.php?t=1130>
 13. Velocity Layer Test Experiment - PianoClack Forum, erişim tarihi Haziran 30, 2025, <https://pianoclack.com/forum/d/1083-velocity-layer-test-experiment?page=3>
 14. What is "round robin" - Instruments Forum - KVR Audio, erişim tarihi Haziran 30, 2025, <https://www.kvraudio.com/forum/viewtopic.php?t=274098>
 15. What is a Round Robin? - Support Centre, erişim tarihi Haziran 30, 2025, <https://spitfireaudio.zendesk.com/hc/en-us/articles/360025864833-What-is-a-Round-Robin>
 16. Unity - Audio Files - Unity - Manual, erişim tarihi Haziran 30, 2025, <https://docs.unity3d.com/353/Documentation/Manual/AudioFiles.html>
 17. Can I produce sound in 48kHz (24bit), or does Unity only work with 44kHz (16bit)?, erişim tarihi Haziran 30, 2025, <https://support.unity.com/hc/en-us/articles/206116296--Can-I-produce-sound-in-48kHz-24bit-or-does-Unity-only-work-with-44kHz-16bit>
 18. Optimizing Audio Assets for Enhanced Performance in Mobile Game Development, erişim tarihi Haziran 30, 2025, <https://moldstud.com/articles/p-optimizing-audio-assets-for-enhanced-performance-in-mobile-game-development>
 19. Ogg Vorbis vs other codecs | Headphone Reviews and Discussion - Head-Fi.org, erişim tarihi Haziran 30, 2025, <https://www.head-fi.org/threads/ogg-vorbis-vs-other-codecs.974560/>
 20. A Guide for Choosing the Right Codec - Audiokinetic, erişim tarihi Haziran 30, 2025, <https://www.audiokinetic.com/en/blog/a-guide-for-choosing-the-right-codec/>
 21. Audio Files - Unity - Manual, erişim tarihi Haziran 30, 2025, <https://docs.unity3d.com/450/Documentation/Manual/AudioFiles.html>

22. After years of trying to design rhythm games around Unity's audio latency, I finally bit the bullet and wrote my own native audio plugin. Our demo is now on Steam! : r/Unity3D - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/Unity3D/comments/101ftlv/after_years_of_trying_to_design_rhythm_games/
23. Optimizing Mobile Game Audio - Number Analytics, erişim tarihi Haziran 30, 2025, <https://www.numberanalytics.com/blog/optimizing-mobile-game-audio-techniques>
24. Adaptive differential pulse-code modulation - Wikipedia, erişim tarihi Haziran 30, 2025, https://en.wikipedia.org/wiki/Adaptive_differential_pulse-code_modulation
25. ADPCM Decoding: - Columbia University Computer Science, erişim tarihi Haziran 30, 2025, <https://www.cs.columbia.edu/~%20sedwards/classes/2004/4840/reports/manic.pdf>
26. 10 Unity Audio Optimisation Tips - Game Dev Beginner, erişim tarihi Haziran 30, 2025, <https://gamedevbeginner.com/unity-audio-optimisation-tips/>
27. Scripting API: AudioSettings.GetDSPBufferSize - Unity - Manual, erişim tarihi Haziran 30, 2025, <https://docs.unity3d.com/ScriptReference/AudioSettings.GetDSPBufferSize.html>
28. Unity Audio Buffer Size -Best Latency vs Best Performance : r/Unity3D - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/Unity3D/comments/1g45kgu/unity_audio_buffer_size_best_latency_vs_best/
29. Unity3D Audio Latency in Android - Stack Overflow, erişim tarihi Haziran 30, 2025, <https://stackoverflow.com/questions/24585027/unity3d-audio-latency-in-android>
30. I am getting a lot of sound latency when developing my game for Android. How can I get my sounds to play immediately? - Unity Support, erişim tarihi Haziran 30, 2025, <https://support.unity.com/hc/en-us/articles/206116316-I-am-getting-a-lot-of-sound-latency-when-developing-my-game-for-Android-How-can-I-get-my-sounds-to-play-immediately>
31. Audio latency | Android NDK, erişim tarihi Haziran 30, 2025, <https://developer.android.com/ndk/guides/audio/audio-latency>
32. Audio latency on Unity 3d Android Platform | by Kunal Tandon | Developer's Arena | Medium, erişim tarihi Haziran 30, 2025, <https://medium.com/developers-arena/audio-latency-on-unity-3d-android-platform-7c22c44dc955>
33. Unity's Android audio latency improvement in 2019.1.0 | by 5argon - Medium, erişim tarihi Haziran 30, 2025, <https://medium.com/@5argon/unitys-android-audio-latency-improvement-in-2019-1-0-ebcffc31a947>
34. Optimize Unity Game Performance — Optimizing audio checklist | by Truong Pham | Medium, erişim tarihi Haziran 30, 2025, <https://medium.com/@truongpham/optimize-unity-game-performance-optimizing-audio-checklist-4dd0fff93be7>

35. Working with Audio Components - Unity Learn, erişim tarihi Haziran 30, 2025, <https://learn.unity.com/tutorial/working-with-audio-components-2019-3>
36. Scripting API: AudioSource - Unity - Manual, erişim tarihi Haziran 30, 2025, <https://docs.unity3d.com/ScriptReference/AudioSource.html>
37. Wwise or FMOD? A Guide to Choosing the Right Audio Tool for Every Game Developer, erişim tarihi Haziran 30, 2025, <https://www.thegameaudioco.com/wwise-or-fmod-a-guide-to-choosing-the-right-audio-tool-for-every-game-developer>
38. Anyone here have worked with FMOD or Wwise? | VI-CONTROL, erişim tarihi Haziran 30, 2025, <https://vi-control.net/community/threads/anyone-here-have-worked-with-fmod-or-wwise.32737/>
39. Wwise vs FMOD - HTML5 Game Devs Forum, erişim tarihi Haziran 30, 2025, <https://www.html5gamedevs.com/topic/12944-wwise-vs-fmod/>
40. Low-latency Multichannel Audio - Sound FX - Unity Asset Store, erişim tarihi Haziran 30, 2025, <https://assetstore.unity.com/packages/audio/sound-fx/low-latency-multichannel-audio-147091>
41. Interactive Audio Dev Platform, Low Latency Audio Engine, Cross Platform Audio | Superpowered, erişim tarihi Haziran 30, 2025, <https://superpowered.com/audio-library-sdk>
42. Design Patterns For Unity Developers: Object Pooling Like A Pro ..., erişim tarihi Haziran 30, 2025, <https://medium.com/@taha.m.gokdemir/design-patterns-for-unity-developers-object-pooling-like-a-pro-f13355a2c21c>
43. Use object pooling to boost performance of C# scripts in Unity, erişim tarihi Haziran 30, 2025, <https://learn.unity.com/tutorial/use-object-pooling-to-boost-performance-of-c-scripts-in-unity?uv=6&projectId=67bc8deaedbc2a23a7389cab>
44. Optimize Unity's Performance with Object Pooling - Wayline, erişim tarihi Haziran 30, 2025, <https://www.wayline.io/blog/optimize-unity-game-performance-with-object-pooling-best-practices-benefits-and-techniques>
45. Typical sheet music, Invention no 1 by Bach. | Download Scientific Diagram - ResearchGate, erişim tarihi Haziran 30, 2025, https://www.researchgate.net/figure/Typical-sheet-music-Invention-no-1-by-Bach_fig3_1885265
46. Method: Bach's 'Invention No. 1' Offers a Glimpse into Baroque Style ..., erişim tarihi Haziran 30, 2025, <https://classicalguitarmagazine.com/method-bachs-invention-no-1-offers-a-glimpse-into-baroque-style-and-ornamentation/>
47. What is the highest pitch that Mozart wrote on piano? : r/classicalmusic - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/classicalmusic/comments/t07xdo/what_is_the_highest_pitch_that_mozart_wrote_on/

48. 40+ Famous Classical Piano Songs (Sheet Music, Tutorials) - Pianote, erişim tarihi Haziran 30, 2025, <https://www.pianote.com/blog/famous-classical-piano-songs/>
49. Fantaisie-Improptu - Wikipedia, erişim tarihi Haziran 30, 2025, <https://en.wikipedia.org/wiki/Fantaisie-Improptu>
50. How to use music in games and applications without violating copyright?, erişim tarihi Haziran 30, 2025, <https://polikarpov.legal/en/blogposts/how-to-use-music-in-games/>
51. Public domain - IMSLP, erişim tarihi Haziran 30, 2025, https://imslp.org/wiki/public_domain
52. International Music Score Library Project - Wikipedia, erişim tarihi Haziran 30, 2025, https://en.wikipedia.org/wiki/International_Music_Score_Library_Project
53. Can you use Royalty Free Music for your game's soundtrack? : r/gamedev - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/gamedev/comments/1ljqxl/can_you_use_royalty_free_music_for_your_games/
54. International Music Score Library Project (IMSLP) - UTA Libraries, erişim tarihi Haziran 30, 2025, <https://libraries.uta.edu/database-use/imslp>
55. IMSLP:Licensing Policy and Guidelines, erişim tarihi Haziran 30, 2025, https://imslp.org/wiki/IMSLP:Licensing_Policy_and_Guidelines
56. How to download scores in MusicXML and MIDI - Newzik blog, erişim tarihi Haziran 30, 2025, <https://en.blog.newzik.com/blog/telecharger-partitions-musique>
57. Why are scores licensed? | Musescore.com, erişim tarihi Haziran 30, 2025, <https://musescore.com/licensing>
58. Musescore User Agreement – Terms of Use, erişim tarihi Haziran 30, 2025, <https://musescore.com/legal/terms>
59. Frequently Asked Questions - MuseScore, erişim tarihi Haziran 30, 2025, <https://musescore.org/en/faq>
60. MIDI - Classical Archives, erişim tarihi Haziran 30, 2025, <https://www.classicalarchives.com/midi.html>
61. vdaron/MusicXml.Net: Quick C# parser for MusicXML - GitHub, erişim tarihi Haziran 30, 2025, <https://github.com/vdaron/MusicXml.Net>
62. MusicXml.NET 3.1.0 - NuGet, erişim tarihi Haziran 30, 2025, <https://www.nuget.org/packages/MusicXml.NET>
63. ikorin24/U8XmlParser: Extremely fast UTF-8 xml parser library - GitHub, erişim tarihi Haziran 30, 2025, <https://github.com/ikorin24/U8XmlParser>
64. Wintergatan/MidiConvert: Convert MIDI into Tone.js-friendly JSON - GitHub, erişim tarihi Haziran 30, 2025, <https://github.com/Wintergatan/MidiConvert>
65. Chart Definition using JSON | Enterprise Architect User Guide - Sparx Systems, erişim tarihi Haziran 30, 2025, https://sparxsystems.com/enterprise_architect_user_guide/17.1/model_publishing/dynchart_with_json.html
66. How do I chart my rhythm game prototype? : r/unity - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/unity/comments/1klm9yt/how_do_i_chart_my_rhythm_g

[ame_prototype/](#)

67. Piano Tiles/Magic Tiles 3 | by Ham Siripanichgon - Medium, erişim tarihi Haziran 30, 2025, <https://hamsiripanichgon.medium.com/piano-tiles-3-d5987f504927>
68. How to Develop Magic Tiles 3 Piano Game App - Cyber Infrastructure, CIS, erişim tarihi Haziran 30, 2025, <https://www.cisin.com/growth-hacks/cost-and-feature-to-develop-software-like-magic-tiles-3-piano-game/>
69. Optimizing audio code for performance and power consumption. - YouTube, erişim tarihi Haziran 30, 2025, <https://www.youtube.com/watch?v=S9vJ-3SF-HM>
70. Optimize your mobile game performance: Tips on profiling, memory, and code architecture from Unity's top engineers, erişim tarihi Haziran 30, 2025, <https://unity.com/blog/games/optimize-your-mobile-game-performance-tips-on-profiling-memory-and-code-architecture-from>
71. Mobile Audio: Challenges and Solutions - GameDev DOU, erişim tarihi Haziran 30, 2025, <https://gamedev.dou.ua/blogs/mobile-audio-challenges-and-their-solutions/?hl=en>
72. Voice Stealing : r/algorithms, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/algorithms/comments/i2j5mj/voice_stealing/
73. Convolution vs Algorithmic Reverb: An In-Depth Comparison - MasteringBOX, erişim tarihi Haziran 30, 2025, <https://www.masteringbox.com/learn/convolution-and-algorithmic-reverb>
74. Convolution vs Algorithmic Reverb? : r/audioengineering - Reddit, erişim tarihi Haziran 30, 2025, https://www.reddit.com/r/audioengineering/comments/1jpprtf/convolution_vs_algorithmic_reverb/
75. Dynamic Eq – Equalizer Plugin - Initial Audio, erişim tarihi Haziran 30, 2025, <https://initialaudio.com/product/dynamic-eq-equalizer-plugin/>
76. Audio Parametric Equalizer Effect - Unity - Manual, erişim tarihi Haziran 30, 2025, <https://docs.unity3d.com/6000.1/Documentation/Manual/class-AudioParamEQEffect.html>
77. Unity 3D / 5 - AUDIO EQUALIZER TUTORIAL (FAST AND EASY) - YouTube, erişim tarihi Haziran 30, 2025, <https://www.youtube.com/watch?v=fOEslX0NhjQ>
78. Sound Effects Scripting - Unity Learn, erişim tarihi Haziran 30, 2025, <https://learn.unity.com/course/beginning-audio-in-unity/tutorial/sound-effects-scripting-1?version=2019.4>
79. Unity - How to do Simple Sound Effect Pitch Randomising - YouTube, erişim tarihi Haziran 30, 2025, <https://www.youtube.com/watch?v=P2Yj6XHsLj0>
80. Optimize your app's size and stay within Google Play app size limits - Play Console Help, erişim tarihi Haziran 30, 2025, <https://support.google.com/googleplay/android-developer/answer/9859372?hl=en>
81. Google Play | Android Developers, erişim tarihi Haziran 30, 2025, <https://developer.android.com/guide/playcore/asset-delivery>